Module 20: Mobile DevOps Fundamentals

- CI/CD Pipelines, Build Automation, Secrets and Signing, Versioning, Environments, and Artifact Repositories
- Enterprise Mobile Application Development Bootcamp, AI-Integrated Edition



©Logical Imagination Group LLC

What We Will Cover

- Part 1: CI/CD Foundations. Pipelines as graphs, reproducible builds and caching, tests and quality gates, secrets, signing, and pipeline security
- Part 2: Releases, Environments, and Artifacts. Versioning and release trains, configuration and promotion, artifact repositories, delivery metrics and the release decision



©Logical Imagination Group LLC

Part 1: CI/CD Foundations: Pipelines, Builds, Tests, and Secrets

- 1 CI/CD Concepts and the Mobile Delivery Pipeline
- 2 Build Automation: Reproducible Builds and Caching
- 3 Automated Testing in the Pipeline and Quality Gates
- 4 Secrets, Signing, and Pipeline Security



©Logical Imagination Group LLC

Lesson 1: CI/CD Concepts and the Mobile Delivery Pipeline

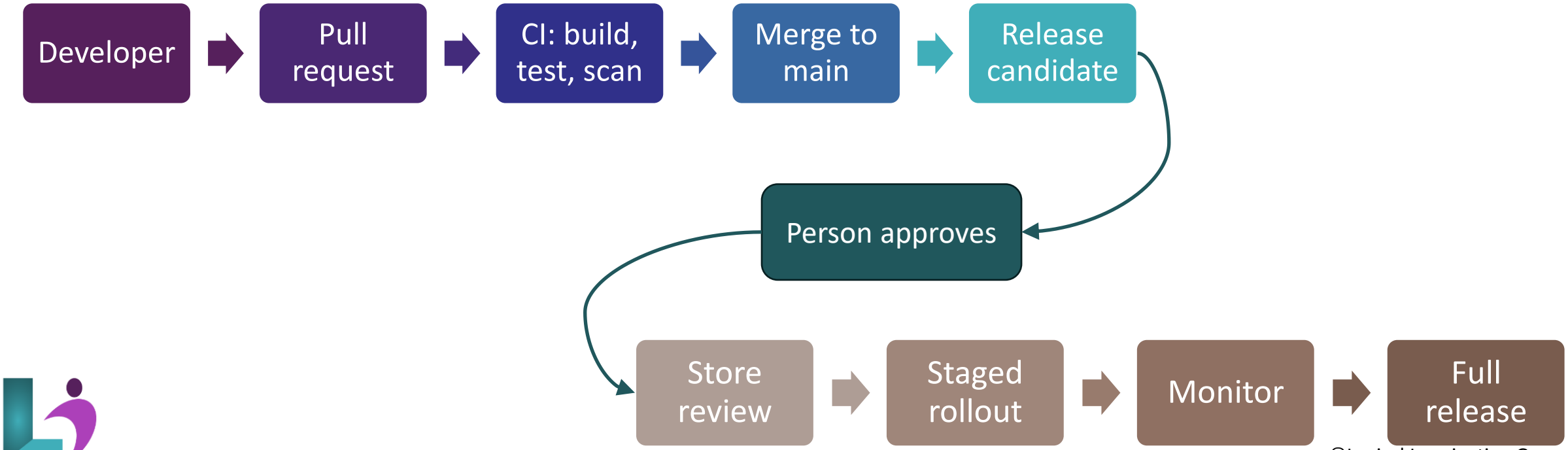
- Explain continuous integration, continuous delivery, and continuous deployment and why mobile differs from web
- Describe the stages of a mobile pipeline and what each one protects
- Model a pipeline as a graph of jobs with dependencies, parallel levels, and path filters
- Read a pipeline's critical path and use it to decide where to speed it up



©Logical Imagination Group LLC

What CI and CD Mean, and Why Mobile Is Different

- Continuous integration: merge small changes often and verify each one automatically
- Continuous delivery: every change that passes is always ready to release; a person decides when
- Continuous deployment: every change that passes is released automatically
- Mobile adds store review, signing, device fragmentation, and users who delay updates



©Logical Imagination Group LLC

One Change, Two Timelines: Web and Mobile

- Web: merge at 10:00, canary by 10:20, all users by 11:00; a bad change is rolled back in minutes
- Mobile: merge at 10:00, release candidate by 10:40, then store review that can take hours to days
- After approval comes a staged rollout over days, and users update when they choose
- A defect that ships can stay on devices for weeks, so prevention beats repair
- Mobile teams compensate with feature flags, server-side kill switches, staged rollouts, and a rehearsed hotfix path



©Logical Imagination Group LLC

The Stages of a Mobile Pipeline

- Validate: lint, static analysis, dependency and secret scans
- Build: compile for each platform and variant; iOS builds need macOS
- Test: unit tests, integration tests, UI tests on emulators or devices
- Package and publish: sign, store the artifact, distribute to testers or the store

Code Bite

```
# Generic shape, not product specific:
validate: [lint, static-analysis,
                  secret-scan]
build:    [android-build, ios-build]
# ios-build needs a macOS runner
test:     [unit, integration, ui-tests]
package:  [sign, publish-artifact]
```



©Logical Imagination Group LLC

Fail Fast: Order Stages by Cost and Failure Rate

- Run cheap, likely-to-fail checks first and expensive ones last
- The same checks in a different order give a very different wait
- A lint error is found at minute 1 instead of minute 37
- Independent checks can also run in parallel

Code Bite

```
# Order A: expensive first
build        10 min
ui-tests     25 min
unit          1 min
lint          1 min   <- fails
# developer learns at minute 37


# Order B: cheapest first
lint          1 min   <- fails
# developer learns at minute 1
# nothing else ran, nothing wasted
```



©Logical Imagination Group LLC

Pipelines as Code: Graphs, Parallelism, and Path Filters

- Define the pipeline in a file that lives with the code and is reviewed like code
- Jobs form a directed graph: a job runs after the jobs it needs
- Independent jobs run in parallel; the critical path sets the minimum time
- Path filters skip jobs a change cannot affect, which matters in monorepos

Code Bite

```
const p = planPipeline([
  {name: "lint", needs: [] }, # 60,000ms
  {name: "unit", needs: [] }, # 180,000ms
  {name: "build", needs: ["lint", "unit"] },
      # 300,000ms
  {name: "deploy", needs: ["build"]}
      # 120,000ms
]);
# levels: [[lint, unit], [build], [deploy]];
# criticalPathMs 600000; totalMs 660000
```



©Logical Imagination Group LLC

Reading a Pipeline File (GitHub Actions Style)

- Triggers say when it runs; path filters limit it to relevant changes
- Each job names the runner it needs; iOS jobs need macOS
- needs creates the edges of the graph
- Syntax differs by product; the ideas do not

Code Bite

```
on:
  pull_request:
    paths: ["android/**", "shared/**"]
jobs:
  lint:
    runs-on: ubuntu-latest
  build:
    needs: [lint]
    runs-on: ubuntu-latest
    steps:
      - run: ./gradlew assembleDebug
# GitHub Actions style; others differ
```



©Logical Imagination Group LLC

Critical Path: A Worked Example

- Earliest finish = a job's duration plus the latest finish among the jobs it needs
- The largest finish time is the critical path
- Slack is how long a job can slip without delaying the run
- Speed up ios by 4 minutes and the run gets faster; speed up android and nothing changes

Code Bite

```
job          min  needs
lint           1  -
unit           3  -
android        8  lint
ios           12  lint
ui-android    15  android
ui-ios        18  ios
sign           2  ui-*, unit
# critical path: ???
```



©Logical Imagination Group LLC

Lesson 1

Skill Check

1. What is the difference between continuous delivery and continuous deployment?

A) None
B) Deployment is slower
C) Delivery keeps every passing change releasable and a person decides; deployment releases it automatically
D) Delivery skips tests

2. Why do iOS builds affect pipeline design?

A) Swift is slow
B) Apple's toolchain needs macOS runners, which are scarcer and costlier
C) They need no tests
D) They cannot be cached

3. What does the critical path of a pipeline tell you?

A) The number of tests
B) The most important job
C) The shortest possible wall-clock time given unlimited parallelism
D) The slowest developer

4. Why order checks from cheapest to most expensive?

A) It looks tidy
B) It saves disk space
C) Tools require it
D) Most failures are reported quickly before costly stages run



©Logical Imagination Group LLC

Lesson 2: Build Automation: Reproducible Builds and Caching

- Explain what makes a build reproducible and why it matters for audit and debugging
- Describe how Gradle and Xcode builds are driven from the command line in a pipeline
- Design a dependency cache with a content-based key and restore keys
- Implement a cache key function and a restore picker



©Logical Imagination Group LLC

Reproducible, Scripted Builds

- The pipeline builds with the same command a developer can run locally
- Pin the toolchain: JDK, Gradle wrapper, Xcode version, and dependency versions
- Commit lock files; avoid floating versions such as latest
- Record the commit, tool versions, and build number with every artifact

Code Bite

```
# Same commands locally and in CI
# (verify flags in current documentation)
./gradlew assembleRelease
# uses the pinned Gradle wrapper
xcodebuild -scheme PNCMobile -destination
   'generic/platform=iOS' archive


Build record stored with the artifact:
  commit 9f2c1ab, build 1087, JDK 17.0.x,
Gradle 8.x, Xcode 16.x
```



©Logical Imagination Group LLC

Why Builds Drift

- Floating versions such as latest or 1.+ pull in tomorrow's code
- Unpinned tools: a runner image update can silently change the JDK or Xcode version
- Build-time clocks, absolute paths, and random IDs end up inside the binary
- Network fetches during a build can return different content each time
- Aim for functional reproducibility first; bit-for-bit identical output is a harder, optional goal

Pro Tips

1. Rebuild a release from six months ago once a quarter, on a clean runner. Every surprise is a gap you found before an auditor or an incident did.



©Logical Imagination Group LLC

Driving Android and iOS Builds from the Command Line

- Android: the Gradle wrapper runs tasks that name what to produce
- iOS: xcodebuild builds, tests, and archives a scheme for a destination
- Publish test results and logs as pipeline artifacts
- Flags and task names can change between releases, so check current documentation

Code Bite

```
# Android
./gradlew test
./gradlew assembleRelease
./gradlew bundleRelease   # .aab

# iOS
xcodebuild test -scheme PNCMobile \
  -destination "$SIM_DEST"
xcodebuild archive -scheme PNCMobile \
  -archivePath build/PNCMobile.xcarchive
# SIM_DEST: set once per runner
```



©Logical Imagination Group LLC

Caching Dependencies and Build Outputs

- Downloading dependencies and recompiling unchanged code dominate many mobile builds
- Cache the dependency directories and, where supported, the build cache
- A cache key must change exactly when the inputs that matter change
- Restore keys let a near match seed the cache when the exact key is missing

Code Bite

```
export function cacheKey(i: KeyInput): string {
  const parts =
      Object.keys(i.files).sort().map((p) => p +
      ":" + sha(i.files[p]));
  // path and content, in a fixed order
  return [i.prefix, i.os, i.toolchain,
      sha(parts.join("\n")).slice(0,
      16)].join("-");
}
// gradle-linux-jdk17-3fa9c2e41b07d5aa; restore
// keys: gradle-linux-jdk17-, gradle-linux-
```



©Logical Imagination Group LLC

Cache Math and Cache Pitfalls

- A cold build downloads for about 6 minutes; a warm cache restores in about 30 seconds
- At 40 runs a day that is roughly 220 runner-minutes saved
- Pitfalls: stale entries, oversized caches, and sharing across unlike OS or toolchain
- A cache is an input you did not review, so treat it as untrusted

Code Bite

```
cold download    6.0 min
cache restore    0.5 min
saved per run    5.5 min
x 40 runs a day  = 220 min
# about 3.7 runner-hours daily


# Cache hygiene
key on content, OS, toolchain
cap the size, expire old entries
never cache secrets or signing files
keep PR caches away from release
```



©Logical Imagination Group LLC

Build Variants, Numbers, and Where Builds Run

- Android build types and flavors and iOS schemes and configurations create debug, staging, and release variants
- Build once per commit and reuse the artifact; do not rebuild for each environment
- Build numbers must increase for every upload; derive them from the pipeline, not from a person
- Choose hosted or self-hosted runners with security, cost, and macOS capacity in mind

Code Bite

Variant	Points at	Signed with	Use
debug	dev services	debug key	developers
staging	test services	test signing	QA and UAT
release	production services	release signing	customers

```
# Build release ONCE per commit;
# promote the same artifact to staging,
# then production.
```



©Logical Imagination Group LLC

Hosted or Self-Hosted Runners?

- Hosted: no maintenance and a quick start, but shared infrastructure and less control
- Self-hosted: control over network, tools, and secrets, but you own patching and capacity
- macOS capacity is the usual iOS bottleneck
- Many banks self-host inside controlled networks

Code Bite

Question	Hosted	Self-hosted
Setup effort	low	high
Patching	vendor	you
Network control	limited	full
Data residency	vendor	yours
macOS capacity	queue	buy Macs
Cost shape	per min	fixed + ops

```
# Prefer fresh (ephemeral) runners:
# one job, then wipe the machine
```



©Logical Imagination Group LLC

Lesson 2

Skill Check

1. What makes a build reproducible?

A) A fast machine
B) Running it twice
C) Pinned toolchain and locked dependencies, so the same source gives the same result anywhere
D) A large cache

2. What should a cache key be based on?

A) The date
B) The developer's name
C) The branch name only
D) A hash of the inputs that determine the cached content, plus OS and toolchain

3. What is a restore key for?

A) Seeding a build from a near-match cache when the exact key is missing
B) Encrypting the cache
C) Deleting old caches
D) Signing the build

4. Why derive build numbers in the pipeline?

A) It is faster to type
B) People forget
C) Stores require every upload to have a higher number, and automation avoids mistakes
D) Numbers must be random



©Logical Imagination Group LLC

Lesson 3: Automated Testing in the Pipeline and Quality Gates

- Describe the test pyramid and where each kind of mobile test runs in a pipeline
- Explain flaky tests, why they erode trust, and how quarantine keeps them visible
- Shard a test suite to cut wall-clock time
- Implement sharding, flaky detection, and a quality gate
- Time: 90 minutes | Lecture, live demo, self-guided exercise, knowledge check



©Logical Imagination Group LLC

The Test Pyramid in a Pipeline

- Many fast unit tests, fewer integration tests, few end-to-end UI tests
- Unit tests run on every commit; UI tests often run on merge or nightly
- Emulators, simulators, and device farms trade cost against realism
- Fast tests belong early so feedback is quick

Code Bite

```
      /\        UI, end-to-end
     /  \       few, slow, nightly
    /----\      Integration
   /      \     some, minutes, fakes
  /--------\    Unit
                many, seconds, every commit
```



©Logical Imagination Group LLC

Which Test Catches Which Defect?

- Pick the cheapest test that can catch the defect
- Pure logic belongs in unit tests
- Boundaries between components belong in integration or contract tests
- Only a real screen, OS, and device can answer some questions

Code Bite

```
Rounding bug in interest calc
  -> unit test (milliseconds)
API renames a response field
  -> integration / contract test
Login crashes on a small screen
  -> UI test, device matrix
New OS changes a permission prompt
  -> UI test, real device
Slow cold start
  -> performance test, real device
```



©Logical Imagination Group LLC

Flaky Tests and Quarantine

- A flaky test passes and fails on the same code
- Flakiness teaches developers to ignore red builds, which is the real damage
- Detect: the same test both failed and passed on the same commit
- Quarantine: stop it blocking merges, keep running and reporting it, assign an owner and a deadline

Code Bite

```
// flaky = failed and passed on the SAME
// commit
export function detectFlaky(history:
    RunRecord[]): string[] {
  // group by commit and test; a group that
  // contains both a pass and a fail is
  // flaky
}
// transfer failed on c2 and passed on c3:
// a different commit, so a real fix, not
// flakiness
```



©Logical Imagination Group LLC

Fixing a Flaky Test

- Common causes: fixed sleeps, shared state, real network calls, test order, clock and timezone
- Wait for a condition, not for a duration
- Reset state in setup and replace network calls with fakes
- Inject the clock so time is under the test's control

Code Bite

```
// Flaky: hopes 500ms is enough
await sleep(500);
expect(screen.hasBalance()).toBe(true);

// Stable: waits for the condition
await waitFor(
  () => screen.hasBalance(),
  { timeoutMs: 5000 });
expect(screen.balance())
  .toBe("$120.00");
```



©Logical Imagination Group LLC

Sharding and Quality Gates

- Sharding splits the suite across parallel runners so wall-clock time falls
- Balance by historical duration, longest first, not by test count
- A quality gate is an automatic decision: coverage, failures, static analysis, and security results
- Gates block on new problems and are tuned so people trust them

Code Bite

```
shardTests([a:90, b:80, c:70, d:60, e:50,
    f:40], 3)
// shard 1: a, f = 130 ms    shard 2: b, e
// = 130 ms    shard 3: c, d = 130 ms
// one runner would need 390 ms; three
// balanced shards need 130 ms
```



©Logical Imagination Group LLC

Greedy Sharding, Step by Step

- Sort tests longest first
- Give each test to the shard with the least work so far
- The slowest shard sets the wall-clock time
- No sharding can beat the longest single test, so split a huge test

Code Bite

```
tests: a90 b80 c70 d60 e50 f40
shards: 3

a90 -> S1   (90)
b80 -> S2   (80)
c70 -> S3   (70)
d60 -> S3   (70 + 60 = 130)
e50 -> S2   (80 + 50 = 130)
f40 -> S1   (90 + 40 = 130)

# slowest shard 130 vs 390 serial
```



©Logical Imagination Group LLC

A Quality Gate in Practice

- A gate lists its rules and reports every failure, not just the first
- Judge new and changed code so old debt does not block everyone
- Ratchet the baseline so coverage never quietly falls
- Allow a logged, time-limited override with a named owner

Code Bite

```
gate: pull-request
  failing tests       == 0
  new-code coverage   >= 80%
  total coverage      >= baseline
  new static issues   == 0
  critical vulns      == 0

result: BLOCKED
  - new-code coverage 62% < 80%
  - 2 new static issues
```



©Logical Imagination Group LLC

Lesson 3

Skill Check ✓

1. Why should most tests be unit tests?

A) They are fast, cheap, and stable
B) They need devices
C) They are more realistic
D) They are required by stores

2. How can you detect a flaky test from history?

A) It fails once
B) It is slow
C) It both failed and passed on the same commit
D) It uses the network

3. How should tests be balanced across shards?

A) By count
B) By historical duration, longest first
C) Randomly
D) Alphabetically

4. What should a quarantined test do?

A) Keep running and reporting without blocking merges, with an owner and deadline
B) Be deleted
C) Block everything
D) Be skipped silently



©Logical Imagination Group LLC

Lesson 4: Secrets, Signing, and Pipeline Security

- Explain why the pipeline is a high-value target and what it holds
- Handle secrets and signing credentials with least privilege and a secret store
- Identify the common pipeline weaknesses: unpinned dependencies, untrusted code, leaked logs, and excess permissions
- Implement a secret scanner, a log redactor, and a pipeline linter
- Time: 75 minutes | Lecture, live demo, self-guided exercise, knowledge check



©Logical Imagination Group LLC

The Pipeline Is a Target

- It holds signing keys, store credentials, and deployment access
- A compromised pipeline can ship malicious code that looks officially signed
- Supply chain attacks reach you through dependencies and build tools
- Treat pipeline configuration as security-sensitive code

Code Bite

```
signing keys, store credentials
 -> sign and ship a malicious build
source code, internal packages
 -> steal or tamper with code
third-party actions and plugins
 -> run attacker code with your secrets
build logs
 -> expose tokens printed by mistake
```



©Logical Imagination Group LLC

Real Pipeline Compromises

- 2020 SolarWinds: attackers altered the build process so trusted, signed updates carried malicious code
- 2021 Codecov: a modified upload script sent customers' CI environment variables, including secrets, to the attacker
- The pattern: the pipeline holds powerful credentials and trusts what it downloads
- Ask of every job: what can it read, what can it sign, and what does it fetch?

Pro Tips

1. Assume any code a job runs can read every secret that job can read. Give each job only what it needs.



©Logical Imagination Group LLC

Secrets and Signing Credentials

- Never commit secrets; keep them in a secret store and inject them only into the jobs that need them
- Scan for secrets before they merge and after they might have leaked; rotate anything exposed
- Android: protect the upload key and use Play App Signing; iOS: protect certificates and provisioning profiles
- Log masking is a safety net, not a plan: never print secrets

Code Bite

```
scanText("password = hunter2hunter2");
// [{ rule: "assigned-secret", line: 1,
// excerpt: "hunt***" }]
scanText("password: ${DB_PASSWORD}");
// []  a reference to the secret store is
// not a secret
redact("token=abcd1234 again abcd1234",
    ["abcd1234"]);
// "token=*** again ***"
```



©Logical Imagination Group LLC

Signing on Android and iOS

- Android: with Play App Signing, Google holds the app signing key and you sign uploads with an upload key
- A leaked upload key can be reset; without Play App Signing, losing the app key means you can never update that app
- iOS: a distribution certificate plus a provisioning profile; protect the private key
- Keep keys in a secrets store, not the repository

Code Bite

```
Android
  upload key -> you sign the upload
  app key    -> Google signs the app
  upload key leaked: request reset

iOS
  certificate + private key
  provisioning profile
    (app ID + team + devices)
  key leaked: revoke, reissue,
  re-sign, resubmit
```



©Logical Imagination Group LLC

Handling Secrets in Jobs

- Use the platform's secret store, never files or variables committed to the repository
- Scope secrets to the environment and job that need them
- Prefer short-lived credentials issued per run over long-lived keys
- Never print secrets; masking helps but is not a guarantee

Code Bite

```
# Bad
echo "key=$API_KEY"       # in the log
API_KEY=abc123 in repo    # in history

# Better
deploy:
  environment: production
  env:
    API_KEY: ${{ secrets.API_KEY }}
# only this job, only on main
# token expires after the run
```



©Logical Imagination Group LLC

Hardening the Pipeline Definition

- Pin third-party actions and plugins to an immutable version, such as a full commit SHA
- Grant the minimum permissions to each job; avoid write-all
- Never run untrusted fork code with secrets
- Do not pipe downloads into a shell; verify checksums and use https

Code Bite

```
permissions:
  contents: read    # minimum for a test job
steps:
  - uses: actions/checkout@0123456789abcdef...
    # pinned to a full SHA (example), use the real one
  - run: ./gradlew test
  # never: - run: curl -sSL
  #   https://example.test/install.sh | bash
  # never: - run: echo ${{ secrets.SIGNING_PASSWORD }}
```



©Logical Imagination Group LLC

Pipeline Lint Rules by Example

- Pin third-party steps to a full commit SHA, not a moving tag or branch
- Never run untrusted pull request code with secrets
- Grant the least permission each job needs
- Never paste untrusted text into a shell command

Code Bite

```
# Risky
uses: some/action@main
permissions: write-all
run: echo "${{ pr.title }}"

# Safer
uses: some/action@3f1c9a7   # SHA
permissions:
  contents: read
env:
  TITLE: ${{ pr.title }}
run: echo "$TITLE"
```



©Logical Imagination Group LLC

Lesson 4

Skill Check ✓

1. Why is a CI pipeline a high-value target?

A) It runs on weekends
B) It stores photos
C) It is slow
D) It holds signing keys and publish access

2. What should you do when a secret has been committed?

A) Revoke and rotate it, because copies may exist
B) Delete the commit
C) Rename it
D) Ignore it

3. Why pin an action to a commit SHA?

A) A tag can be moved to different code; a SHA cannot
B) Shorter names
C) It runs faster
D) It saves cache

4. Why is log masking not enough?

A) It is illegal
B) It may miss transformed or partial values; the control is never to print secrets
C) It deletes logs
D) It is slow



©Logical Imagination Group LLC

Part 2: Releases, Environments, and Artifacts

- 5 Release Pipelines and Versioning (90 min)
- 6 Environment Management and Configuration (75 min)
- 7 Artifact Repositories and Promotion (75 min)
- 8 Release Readiness Workshop: Metrics, Gates, and Human Oversight (105 min)



©Logical Imagination Group LLC

Lesson 5: Release Pipelines and Versioning

- Compare trunk-based development with release branches for mobile apps
- Apply Semantic Versioning and derive store build numbers that always increase
- Generate a version bump and a change log from Conventional Commits
- Implement a semver parser, comparator, version code function, and change log generator
- Time: 90 minutes | Lecture, live demo, self-guided exercise, knowledge check



©Logical Imagination Group LLC

Branching and Release Trains

- Trunk-based development: short-lived branches merged to main many times a day, with feature flags hiding unfinished work
- Release branches: cut a branch to stabilize a release while main moves on
- Mobile favors a release train: a regular schedule, with fixes cherry-picked to the release branch
- Whatever the model, every merge must keep main releasable

Code Bite

```
main:  o--o--o--o--o--o--o--o--o--o
       (always releasable; flags hide
       unfinished work)
                \
release/2.4:     o--f1--f2
       (fixes only, cherry-picked
       from main)
                          |
                    tag v2.4.0
 -> store review
 -> staged rollout
```



©Logical Imagination Group LLC

A Two-Week Release Train

- A train leaves on a schedule; work that is not ready catches the next one
- A release branch accepts fixes only
- Predictable dates beat heroic deadlines
- Hotfixes branch from the released tag

Code Bite

```
Wk1 Mon   cut release/2.5 from main
Wk1 Tue   RC1 built, QA starts
Wk1 Thu   regression fixes only
Wk2 Mon   RC2, UAT, security review
Wk2 Tue   submit to the stores
Wk2 Thu   staged rollout 5% -> 25%
Wk2 Fri   go/no-go for 100%

# hotfix: branch from tag v2.5.0
# fix, build, rollout, merge back
```



©Logical Imagination Group LLC

Semantic Versioning and Store Build Numbers

- Semantic Versioning: MAJOR.MINOR.PATCH, with optional pre-release such as 2.0.0-rc.1
- Increment MAJOR for breaking changes, MINOR for compatible features, PATCH for compatible fixes
- Android versionCode is an integer that must increase with every upload, up to 2,100,000,000; versionName is the text users see
- iOS shows a marketing version and a separate build number that must increase for each upload of a version

Code Bite

```
// major * 1,000,000 + minor * 10,000 +
// patch * 100 + build (each of minor,
// patch, build 0 to 99)
versionCode(parseSemver("3.12.4"), 7)
// 3120407
versionCode(parseSemver("4.0.0"))
// 4000000   always greater than any 3.x.y
// code
```



©Logical Imagination Group LLC

Versions and Build Numbers in Practice

- Marketing version 2.5.1: MAJOR for breaking changes, MINOR for features, PATCH for fixes
- Android: versionName is the label; versionCode is an integer that must increase on every upload
- iOS: CFBundleShortVersionString is the label; CFBundleVersion is the build number
- Derive build numbers from a pipeline counter or from the version, never by hand

Pro Tips

1. A formula such as major x 10000 + minor x 100 + patch turns 2.5.1 into 20501. Keep the formula in code, and mind that Android caps versionCode at 2.1 billion.



©Logical Imagination Group LLC

Conventional Commits, Automatic Bumps, and Change Logs

- Conventional Commits give each message a type: feat, fix, perf, docs, chore, and so on
- A ! after the type or a BREAKING CHANGE footer marks a breaking change
- The highest-impact commit since the last release decides the bump
- A change log can be generated from the same messages and reviewed by a person

Code Bite

```
feat(login): add passkey sign-in -> minor
fix: crash when the list is empty -> patch
feat!: drop iOS 15 support -> major
docs: update README -> no release

Change log:
## 2.0.0
### Breaking changes
- drop iOS 15 support
### Features
- **login**: add passkey sign-in
```



©Logical Imagination Group LLC

From Commits to Version to Change Log

- Commit prefixes tell the tool how big a change is
- Highest rank wins: breaking change, then feat, then fix
- Group commits into change log sections
- Review generated notes before publishing; customers read them

Code Bite

```
Since v2.4.0:
  fix: crash on empty statement
  feat: add biometric login
  fix: wrong date in receipt
-> feat, no breaking: MINOR bump
-> 2.4.0 becomes 2.5.0
## 2.5.0
### Features
- add biometric login
### Fixes
- crash on empty statement
- wrong date in receipt
```



©Logical Imagination Group LLC

Lesson 5

Skill Check ✓

1. Which change requires a MAJOR bump in Semantic Versioning?

A) A breaking change
B) A bug fix
C) A documentation edit
D) A backward-compatible feature

2. What must be true of Android's versionCode?

A) It can be reused
B) It is shown to users
C) It is a positive integer that must increase with each release, up to 2,100,000,000
D) It must be a string

3. Which is lower in precedence?

A) 1.0.0
B) They are equal
C) 1.0.0-rc.1
D) Cannot compare

4. Why should a person still review automated release notes?

A) Notes are not allowed
B) Machines cannot read
C) It is faster
D) Commit messages are written quickly and customers need accurate, plain language



©Logical Imagination Group LLC

Lesson 6: Environment Management and Configuration

- Describe the environments an enterprise mobile release passes through and what each one is for
- Build once and promote one artifact, with configuration applied per environment
- Layer and validate configuration, and keep secrets out of app-bundled files
- Implement config merging, validation, secret detection, and a promotion check
- Time: 75 minutes | Lecture, live demo, self-guided exercise, knowledge check



©Logical Imagination Group LLC

Environments and What Each Is For

- Development: fast, unstable, safe to break
- Test or QA: stable enough for automated and manual testing with test data
- Staging or pre-production: mirrors production for final verification
- Production: real customers, real money; change is controlled

Code Bite

```
dev -> test/QA -> staging -> production


dev:       fast, fake data, anyone
test/QA:   stable, synthetic data, QA + CI
staging:   mirrors prod, masked or
           synthetic data, release team
prod:      controlled, real customers,
           change approval
```



©Logical Imagination Group LLC

The Environment Ladder

- Each rung answers a different question
- Lower rungs are cheap and disposable; upper rungs resemble production
- Mobile distribution channels map onto the upper rungs
- Production data stays in production

Code Bite

```
dev       mock services
qa        synthetic data
staging   prod-like, masked data
uat       business sign-off
prod      real customers, real data

# Mobile channels
TestFlight / Play internal track
  -> staging and UAT audiences
store release, staged rollout
  -> prod
```



©Logical Imagination Group LLC

Build Once, Promote, and Layer Configuration

- Build one artifact per commit and promote that same artifact through the environments
- Rebuilding for each environment means production runs code that was never tested
- Configuration is layered: defaults, then environment values, then overrides
- Validate configuration at build or start-up: required keys, types, allowed values, and no unknown keys

Code Bite

```
const config = mergeConfig(defaults, envProd,
    overrides);
// later layers win; objects merge
const errors = validateConfig(config, schema);
// required, types, allowed values, unknown keys
if (errors.length) throw new
    Error(errors.join("\n"));
checkPromotion([{ env: "test", artifactDigest:
    "sha256:aaa" }, { env: "prod",
    artifactDigest: "sha256:aaa" }]);
```



©Logical Imagination Group LLC

Layered Configuration with Validation

- Later layers override earlier ones: defaults, then environment, then run-time values
- Validate the merged result before the app or pipeline uses it
- Rules catch the expensive mistakes
- Fail the pipeline, not the customer

Code Bite

```
defaults: { timeoutMs: 10000 }
staging:  { apiUrl: "https://stg.x" }
prod:     { apiUrl: "https://api.x",
            debug: false }

merged(prod) = {
  timeoutMs: 10000,
  apiUrl: "https://api.x",
  debug: false }

# rules: https only, debug off,
# no test hosts in prod
```



©Logical Imagination Group LLC

Secrets Stay Out of the App; Flags Control Exposure

- Anything inside the app bundle can be extracted, including keys and passwords
- Keep secrets on the server and issue short-lived tokens to the app
- Feature flags decouple deploying code from exposing a feature
- Flags need owners and removal dates, and their configuration needs the same review as code

Code Bite

```
bundledSecrets({ auth: { clientId: "mobile-
    app", clientSecret: "abc123",
    tokenEndpoint: "https://t.test/token" }
    });
// ["auth.clientSecret"]   clientId is
// public and tokenEndpoint is an address,
// so they are fine
```



©Logical Imagination Group LLC

The Feature Flag Lifecycle

- Flags separate deploying code from exposing it
- A kill switch turns off a bad feature without a release
- Roll out by percentage or cohort and watch the metrics
- Flag debt is real; remove flags once a feature is fully on

Code Bite

```
create    flag off, code merged
test      on for staff only
roll out  1% -> 10% -> 50% -> 100%
watch     crashes, errors, latency
retire    delete flag and old path


// in the app
if (flags.isOn("new-transfer-ui")) {
  showNewTransfer();
}
```



©Logical Imagination Group LLC

Lesson 6

Skill Check ✓

1. What does build once and promote mean?

A) Build in production
B) Promote developers
C) Build only on Fridays
D) One artifact moves through environments with only configuration changing

2. Why are secrets in app configuration not secret?

A) They are too long
B) Anyone with the app can extract its bundled files
C) They expire
D) They are encrypted

3. Why reject unknown configuration keys?

A) To save space
B) To slow builds
C) Required by stores
D) So typos fail loudly instead of silently using defaults

4. What does a feature flag decouple?

A) Builds from commits
B) Teams from managers
C) Code from tests
D) Deploying code from exposing a feature to users



©Logical Imagination Group LLC